MACHINE LEARNING UNIVERSITY

Kiro IDE – Lab Administrator Setup Guide

Guide target personas:

- Lab administrator - Responsible for setting up and validating the Kiro IDE environment for EEP Agentic AI course labs
- Faculty - Leads the course sessions and needs a fully configured Kiro IDE to demonstrate features.
- Course participant - Learner completing hands-on agentic AI labs who needs Kiro IDE installed on a local machine

Note: The lab administrator can be the faculty owning the course with lab hands-on activities, if they have the correct information and permission to create and setup Kiro IDE.

A Step-by-Step Guide to Setup and Access Kiro IDE

Before You Start

Download the Kiro installer from the official downloads page: https://kiro.dev/downloads/

- Select the installer that matches the participant's operating system. Detailed installation instructions are available at: https://kiro.dev/docs/getting-started/installation/

Getting Started with Kiro IDE

Kiro IDE is an AI-powered Integrated Development Environment built by Amazon, designed to support agentic AI workflows. Unlike traditional coding assistants, Kiro uses specs, hooks, steering files, and MCP server integrations to enable autonomous, requirement-driven software development. Students will be using it in their hands-on labs for their EEP Agentic AI course.

Admin First Time Setup

When Kiro launches for the first time, a setup wizard guides you through the initial configuration. Complete all steps below before beginning any EEP Agentic AI lab activities.

Step 1 – Authenticate with AWS Builder ID

To sign in with AWS Builder ID:

1. In Kiro, choose Login with AWS Builder ID. You will be redirected to your default web browser to complete the sign in process.
2. Enter your email address and then choose Next.
3. Enter your password and then choose **Sign in**.
4. Choose Allow access to authorize the Kiro App.

For additional authentication information or to explore other authentication methods, please access this resource: Kiro documentation.

> 💡 **Note:** Authentication requires an active internet connection. Kiro uses secure OAuth 2.0 to authenticate via your browser — your credentials are never stored locally in plain text.

Step 2 – Import VS Code settings

Because Kiro is built on the VS Code open-source foundation, it can import your existing VS Code settings, key bindings, and extensions for a familiar experience.

1. When prompted, select **Import from VS Code** to automatically migrate settings.
2. Review the imported settings and extensions list. Click **Confirm Import** to apply.

If you prefer a clean setup, select **Start Fresh** to skip the import.

Step 3 – Select the Theme

1. Kiro presents several built-in theme options during first-time setup (e.g., Dark, Light, High Contrast).
2. Select your preferred theme and click **Apply**.
3. You can change the theme later at any time via **File → Preferences → Color Theme** (Windows/Linux) or **Kiro → Preferences → Color Theme** (macOS).

Step 4 – Configure Shell Integration

Shell integration allows Kiro's terminal to work seamlessly with your system shell (bash, zsh, PowerShell, etc.).

1. When prompted to configure shell integration, click **Enable Shell Integration**.
2. Restart your terminal (or open a new terminal tab) within Kiro to apply the changes.

3. Verify shell integration is active by opening the integrated terminal (**Ctrl+`** or **Cmd+`** on macOS) and checking that your shell prompt appears correctly.

> 💡 **Tip:** Shell integration can also be configured manually later. Open the Command Palette (**Ctrl+Shift+P** / **Cmd+Shift+P**) and search for **Terminal: Install Shell Integration**.

Validate your Setup

Before the lab session begins, confirm that Kiro is fully functional by completing the following validation checks:

#	Validation Check	Expected Result
1	Kiro launches without errors	IDE opens to welcome/start screen
2	Builder ID shown in status bar (bottom-left)	Your display name or email is visible
3	Open integrated terminal (Ctrl+` / Cmd+`)	Terminal opens with working shell prompt
4	Open Kiro Chat panel (click chat icon in sidebar)	Chat panel opens; AI responds to a test message
5	Create or open a project folder	Workspace loads; file explorer is visible

Setup recommended solutions

1. Kiro launches without errors

If Kiro fails to launch:

- Check your internet connection
- Verify authentication credentials are valid
- Try restarting Kiro
- On Mac: If you see "App can't be opened," go to **System Settings → Privacy & Security** and click **Open Anyway**, or run: sudo xattr -rd com.apple.quarantine /Applications/Kiro\ IDE.app
- On Windows: If antivirus blocks it, temporarily disable or add Kiro to allow list

2. Builder ID shown in status bar (bottom-left)

If your Builder ID isn't visible:

- Sign out and sign back in
- Verify your authentication method is active
- Check the status bar for error indicators

3. Open integrated terminal (Ctrl+ / Cmd+)

If terminal hangs or won't open:

- **Quick fix for Mac**: Open Command Palette (Cmd+Shift+P) → **Kiro: Enable Shell Integration**
- Add this to your ~/.zshrc (or ~/.bashrc):

bash

```
if [[ "$TERM_PROGRAM" != "kiro" ]]; then
  # your shell customizations here
fi
```

- Check default shell settings in Kiro preferences
- Try using menu: **Terminal → New Terminal**

4. Open Kiro Chat panel (click chat icon in sidebar)

If chat panel won't load or shows errors:

- **Change settings to AUTO** in Kiro IDE chat box
- If you have multiple Kiro instances open (more than 2), close extras and **reboot your laptop**
- For Remote SSH on Amazon Linux 2 with stuck loading:

bash

```
ls -lt ~/.kiro-server/bin/
# Use the latest version hash
/opt/vsc-patchelf/bin/patchelf --set-rpath /opt/vsc-sysroot/lib ~/.kiro-server/bin/<VERSION>/extensions/kiro.kiro-agent/node_modules/@lancedb/vectordb-linux-x64-gnu/index.node
```

Then restart Kiro

- **Clean slate approach** if above doesn't work:

bash

```
pkill -f kiro
rm -rf ~/.kiro-server
```

Then restart Kiro

5. Create or open a project folder

If workspace won't load:

- Check folder permissions
- Verify the folder path exists and is accessible
- Try **File → Open Folder** from menu
- For Brazil workspaces: Run Command Palette → **Setup (generate) java classpath -- Powered by Bemol**
- Enable **Viceroy → Config: Include Workspace As Folder** setting

General Troubleshooting Tips

- Check logs: **View → Output** → Select relevant log source
- Restart Kiro after making configuration changes
- Ensure you're running the latest version
- Verify internet connectivity for AI features

After Sign-Up: Next Steps

Once Kiro is installed and authenticated, familiarize participants with the following core features used throughout the EEP Agentic AI labs:

📄 Specs

Specs are Kiro's requirement documents for agentic tasks. A spec defines **what** you want to build by describing requirements, and Kiro's agent uses it to drive implementation automatically.

- Create a spec from the Command Palette or the Kiro sidebar
- Specs are stored as Markdown files in the **.kiro/specs/** directory of your project
- The agent generates tasks, code, and tests based on the spec requirements

🦫 Hooks

Hooks are event-driven automations that trigger Kiro actions when certain file system events occur (e.g., file save, file creation). They allow the IDE to autonomously react to your workflow.

- Hooks are configured in **.kiro/hooks/** as YAML files
- Example: Automatically update tests when source files change
- Example: Run a linter or formatter on every file save

💬 Chat

The Chat panel provides a conversational interface to Kiro's AI assistant. Use Chat to ask questions, get code explanations, generate snippets, and debug issues interactively.

- Access Chat from the left sidebar or via **Ctrl+Shift+I** / **Cmd+Shift+I**
- Chat is context-aware: it can see your open files and workspace
- Use **@workspace**, **@file**, or **@terminal** to provide specific context

🧭 Steering

Steering files are persistent instructions that guide Kiro's AI behavior across your entire project. They set coding standards, architectural patterns, and custom rules that the agent will follow in every interaction.

- Steering files are stored in **.kiro/steering/** as Markdown files
- Use steering to enforce project conventions (e.g., *always use TypeScript*, *prefer functional patterns*)
- Steering instructions are automatically included in every agent prompt

🔌 MCP Servers

Kiro supports the **Model Context Protocol (MCP)**, which allows you to connect external tools and data sources to Kiro's AI agent. MCP servers extend the agent's capabilities beyond the local IDE.

- Configure MCP servers in the Kiro settings under **MCP Servers**
- Examples: Connect to a database, web search tool, or custom API
- MCP tools appear as callable functions in Chat and agent tasks

> 💡 **Getting started with your first project:**
>
> Follow the official Kiro first-project tutorial for a hands-on walkthrough: https://kiro.dev/docs/getting-started/first-project/

Helpful Resources

Resource	Description	Link
Kiro Downloads	Official download page for macOS, Windows, and Linux installers	kiro.dev/downloads/
Kiro Documentation	Full official documentation covering all Kiro features and concepts	kiro.dev/docs/
Installation Guide	Step-by-step installation instructions for all supported platforms	Getting Started
First Project Tutorial	Hands-on tutorial for building your first project with Kiro AI features	First Project
Kiro FAQs	Answers to common questions about Kiro setup, features, and billing	kiro.dev/faq/
AWS Builder ID Sign-Up	Create a free AWS Builder ID for Kiro authentication	profile.aws.amazon.com

🛠️ Troubleshooting: Common Beginner Issues

If participants encounter issues during setup, refer to the following common resolutions:

❌ Problem: Kiro won't launch after installation

✅ **Fix**:

- macOS: Check **System Settings → Privacy & Security** and click **Open Anyway**
- Windows: Right-click the installer and select **Run as Administrator**
- Ubuntu: Ensure dependencies are installed with **sudo apt-get install -f**

❌ Problem: Authentication fails or times out

✅ **Fix**:

- Confirm the machine has an active internet connection
- Try opening the auth URL manually in a browser and signing in
- Check that pop-ups are not blocked in your browser
- Sign out and sign back in: click your profile name in the status bar → **Sign Out**

❌ Problem: Chat / AI features are unresponsive

✅ **Fix**:

- Verify you are authenticated (Builder ID shown in status bar)
- Check internet connectivity
- Restart Kiro and try again

Note: For more information, please consult the FAQs at https://kiro.dev/faq/